ПРОЕКТ-ТЕОРИЯ КАК РАБОТАЕТ

Полное сопоставление теоретического PDF на 1499 страниц с реализацией EH/Unwind Analyzer: что теория требует от анализатора, что проект делает, где это находится в коде, что реализовано частично и как это защищать на комиссии.

Параметр	Значение
Исходная теория	C:\Users\Templar\Desktop\итогоая теория.pdf
Проанализировано страниц	1499
Найдено нумерованных пунктов	785
Кандидатных предложений с алгоритмическими требованиями	10997
Проект	C:\Users\Templar\Desktop\вкр\ВКР ПРОЕКТ ОКОНЧЕННЫЙ\ip_full
Цель документа	Не переписать теорию, а показать, какие требования из нее превращены в работающий проект и где проходят границы реализации.

1. Как теория учитывалась полностью

Главная ошибка предыдущего варианта была в том, что он слишком сильно смотрел на блок RUNTIME_FUNCTION и недостаточно показал остальную книгу. В этой версии теория рассматривается не по одному термину и не по одному разделу, а как полный маршрут: exception event -> machine context -> PE image -> section/data directories -> .pdata -> RUNTIME_FUNCTION -> .xdata -> UNWIND_INFO -> handlers -> static findings -> dynamic validation -> report/evidence -> testing/scope.

При чтении использовались не только слова parser, static, dynamic или analyzer. Учитывались абзацы, где теория описывает алгоритм без явного слова 'проект': например 'сначала проверить NumberOfRvaAndSizes', 'RVA нельзя читать как file offset', 'Size должен быть кратен 12', 'finding должен объяснять rule/observed/impact', 'dynamic не заменяет static'. Такие места были сведены в тематические требования.

Блок теории	Страницы	Как он влияет на проект
Часть I-A	стр. 5-60	Связная лекционная версия глав 1-6: общий маршрут от исключения до unwind-анализатора.
Глава 1	стр. 7-16 / 62-107	Исключение как runtime-событие, машинный контекст, стек, handler, search/unwind phase.
Глава 2	стр. 17-25 / 108-165	PE как карта загруженного образа: headers, секции, RVA/VA/file offset, Data Directory, Exception Directory.
Глава 3	стр. 26-33 / 166-241	.pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO как карта функций и ссылка на unwind metadata.
Глава 4	стр. 34-42 / 242-416	Prolog, epilog, UNWIND_CODE, ABI, nonvolatile registers и смысл unwind codes.
Глава 5	стр. 43-50 / 417-543	RtlLookupFunctionEntry, RtlVirtualUnwind, caller context, failure modes stack walking.
Глава 6	стр. 51-59 / 544-721	First/second chance, VEH/SEH/UEF, C++ exceptions, границы static/dynamic анализа.
Глава 7	стр. 723-866	Практическая ручная модель анализа: как идти по слоям и превращать знания в проверяемый маршрут.
Глава 8	стр. 867-1035	Аномалии, обфускация, dynamic function tables, JIT, runtime-возможности и ограничения.
Глава 9	стр. 1036-1151	Архитектура анализатора: pipeline от недоверенного PE к evidence map, declared/observed/runtime models.
Глава 10	стр. 1152-1223	Сборка модели Windows x64 exception/unwind, validation/corpus/testing логика.
Глава 11	стр. 1224-1481	Архитектура EH/Unwind Analyzer, файлы, report model, режимы, evidence и practical delivery.
Глава 12	стр. 1482-1495	Каталог static findings, evidence families, почему 27, критерии включения finding-ов.

Что считается прямым требованием, а что фоном

Не каждая страница теории обязана стать функцией в C. Часть текста объясняет Windows-механику и нужна для защиты архитектуры. Прямым требованием считалось то, что задает действие анализатора: что читать, что проверять, где остановиться, где продолжать с warning, как интерпретировать результат, какие evidence выводить. Фоновым материалом считалось объяснение, почему Windows устроена именно так. Но даже фон не выброшен: он отражен в обосновании архитектуры и ограничений.

Итоговая карта тематических попаданий

Тема	Страниц в теории	Смысл для ВКР
Windows exceptions/runtime model	883	runtime-событие, handler route, first/second chance
PE headers/data directories	485	парсер PE32+, Data Directory, Exception Directory
RVA/VA/file offset	1016	переводы адресов и запрет читать виртуальный zero-fill как файл
Sections/bounds/permissions	1201	границы raw/virtual, права секций, placement
RUNTIME_FUNCTION/.pdata	1061	function table как карта code range -> unwind metadata
UNWIND_INFO/UWOP	1099	декодирование рецепта восстановления кадра
Handlers/language data	930	EHANDLER/UHANDLER, handler RVA, limits C++ EH
Static findings/evidence	1201	27 findings, score, confidence, report proof
Dynamic validation	1260	Debug API, handler hits, VEH/VCH/UEF, dynamic tables
Validation/testing	520	корпус, negative cases, smoke/regression logic
Architecture/CLI/build	695	parser/static/dynamic/report/modules
Limits/defense	1148	что проект не делает и почему это честно

2. Главная матрица требований: теория -> проект -> код

Эта матрица - не краткое оглавление. Она показывает, какие конкретные требования к работе анализатора были извлечены из всей теории и где они отражены в коде. После матрицы каждый крупный блок подробно разобран человеческим языком: зачем проверка нужна, как проект ее выполняет технически, какой вывод делает и где границы реализации.

ID	Требование из теории	Суть	Код	Статус
T01	Не доверять входному PE как истине	Файл считается источником заявлений, которые надо проверять по границам, размерам и связям.	src/util.c:41-68; src/pe.c:35-146	Реализовано: bounded reads, PE signature/header checks, fatal errors для неподдерживаемой основы.
T02	Разделять file, image и live module	RVA, VA и file offset нельзя смешивать; static читает файл, dynamic работает с actual ImageBase процесса.	src/pe.c:264-299; src/dynamic_win.c:789-803	Реализовано: static RVA->file offset; dynamic base_main + RVA.
T03	Поддерживать именно Windows x64 PE32+	Теория EH/unwind относится к x64 модели с .pdata/.xdata и 32-bit RVA внутри PE32+.	include/pe.h:18-19; src/pe.c:67-88	Реализовано как осознанный scope, PE32/x86 не смешивается с x64 unwind.
T04	Сначала пройти mandatory PE layer	DOS header, e_lfanew, NT signature, COFF, Optional Header, Magic, SizeOfImage, SizeOfHeaders.	src/pe.c:35-146	Реализовано; при фатальных нарушениях дальнейший анализ не строится.
T05	Проверять Data Directory count	Exception Directory нельзя читать, если NumberOfRvaAndSizes не покрывает нужный индекс.	src/pe.c:121-134	Реализовано: недостаточный directory count фиксируется как parse problem.
T06	Корректно понимать отсутствующий Exception Directory	VA=0 и Size=0 - отсутствие таблицы; частично нулевая пара подозрительна.	src/pe.c:230-235; src/unwind.c:623-626	Реализовано: zero/zero не авария, partial zero - malformed.
T07	Сопоставлять Exception Directory с image и файлом	Directory RVA/Size должны быть внутри SizeOfImage и мапиться в реальные bytes файла для static чтения.	src/pe.c:230-235; src/unwind.c:631-647	Реализовано: OOB/mapping/size checks до чтения entries.
T08	Проверять section table как основу координат	Raw/virtual ranges, overlaps, alignment и SizeOfImage определяют доверие к дальнейшим RVA.	src/pe.c:174-228; src/heuristics.c:789-829	Реализовано: первые 6 findings покрывают PE layout.
T09	Учитывать headers и zero-fill	RVA может попадать в headers; virtual tail может существовать в памяти, но не в файле.	src/pe.c:276-299	Реализовано: headers мапятся отдельно, virtual tail не читается как raw bytes.
T10	Читать RUNTIME_FUNCTION как три little-endian DWORD	BeginAddress, EndAddress, UnwindInfoAddress - RVA, запись 12 байт.	src/util.c:53-68; src/unwind.c:657-659	Реализовано: чтение DWORD bounded и little-endian через util.
T11	Проверять размер Exception Directory	Size должен давать целое число RUNTIME_FUNCTION entries; некратность 12 - тревожный сигнал.	src/unwind.c:639-647	Реализовано: too small / not multiple становятся parse issue.
T12	Проверять диапазоны функций	BeginAddress < EndAddress, границы должны быть внутри image, таблица должна быть однозначной.	src/heuristics.c:612-779; src/heuristics.c:850-887	Реализовано: invalid_ranges, rf_oob, rf_unsorted, rf_overlaps, duplicate_begin.
T13	Не считать gaps автоматической ошибкой	Дыры между RUNTIME_FUNCTION могут быть leaf code или padding.	src/heuristics.c:612-779	Реализовано осторожно: gaps не являются отдельным finding.
T14	Проверять UnwindInfoAddress	Это RVA на UNWIND_INFO; он должен быть внутри image и мапиться в файл при static чтении.	src/unwind.c:55-68; src/heuristics.c:865-868	Реализовано: unwind_rva_oob и parse failure.
T15	Разбирать UNWIND_INFO по grammar Windows x64	Version, Flags, CountOfCodes, slots, operands, padding, tail должны читаться без усечения.	src/unwind.c:164-604	Реализовано: parser всех основных UWOP и ошибок усечения.
T16	Различать handler flags и CHAININFO	EHANDLER/UHANDLER ведут к handler RVA; CHAININFO ведет к chained RUNTIME_FUNCTION.	src/unwind.c:215-220; src/unwind.c:522-548	Реализовано: конфликт flags, chain tail, handler tail.
T17	Строить declared model stack effects	UNWIND_CODE должен давать смысл: stack allocation, saved nonvolatile registers, machine frame.	src/unwind.c:274-520	Частично реализовано: считает fixed_stack_bytes и masks; полного disasm сравнения нет.
T18	Сравнивать metadata placement с ролью секций	Unwind metadata обычно read-only/non-executable, handler target должен быть executable.	src/heuristics.c:117-153; src/heuristics.c:966-997	Реализовано static по section characteristics, не runtime page protections.
T19	Искать reuse/dominance patterns	Массовое повторение UNWIND_INFO/handler targets может быть обфускационным или packer-like сигналом.	src/heuristics.c:681-787; src/heuristics.c:909-955	Реализовано через ratios, thresholds, dominant RVAs.
T20	Использовать correlation, а не одиночный шум	Сильнее не отдельный слабый сигнал, а связка table/unwind/placement/reuse/dynamic.	src/heuristics.c:1004-1023; src/report.c:507-518	Реализовано: combo findings и static/dynamic correlation.

ID	Требование из теории	Суть	Код	Статус
T21	Объяснять finding как evidence	Finding должен говорить где найдено, что нарушено, что наблюдалось, как должно быть и почему важно.	src/report.c:65-195; src/report.c:1347-1365	Реализовано: Rule/Observed/Expected/Impact в HTML/JSON.
T22	Не делать malware verdict	Теория говорит о suspicious evidence, а не о доказательстве вредоносности.	src/heuristics.c:503-587; docs/evidence_matrix.md	Реализовано: suspicion score/confidence, не антивирусный verdict.
T23	Наблюдать runtime handler route	Dynamic должен увидеть регистрации, handler hits, exceptions и function table registrations.	include/dynamic.h:3-13; src/dynamic_win.c:656-1195	Реализовано через Windows Debug API и breakpoints.
T24	Учитывать dynamic function tables/JIT	Static не увидит таблицы, созданные в памяти; dynamic должен хотя бы фиксировать регистрацию.	src/dynamic_win.c:569-573; src/dynamic_win.c:886-1029	Частично реализовано: фиксируются API calls, но не реконструируются все remote entries.
T25	Показывать runtime evidence отдельно от static score	Динамика не тот же score 0..100; это observed runtime signals и runtime evidence level.	src/report.c:486-604; src/report.c:1825-1885	Реализовано: dynamic_signals, runtime_evidence_level, final assessment.
T26	Поддерживать static-only, dynamic-only, mixed запуск	Разные режимы нужны для файла на диске, runtime проверки и объединенного evidence.	src/main.c:191-242; src/main.c:389-408	Реализовано: --heuristics, --dynamic и совместное использование.
T27	Оставлять ручную проверку там, где нужна семантика кода	Prolog disassembly, C++ scope tables и полный stack walk шире текущей реализации.	src/report.c:624-677; src/report.c:1807-1822	Честно оформлено: report дает manual validation steps, а не делает ложный all-seeing вывод.

3. Полный учет глав теории: что взято из каждой части

Здесь отдельно показано, что теория учитывалась не только по третьему блоку. Для каждой крупной части указано, какую инженерную нагрузку она несет, где это отражено в проекте и какая граница реализации остается честной. Это важно для защиты: комиссия может открыть любую главу теории и спросить, как она связана с проектом.

Часть I-A и глава 1: exception как runtime-сцена (стр. 5-16 и 62-107)

Смысл для анализатора: Эта часть задает не структуру файла, а правильную точку зрения: исключение происходит в конкретном потоке, с конкретным RIP/RSP/register context и дальнейшим маршрутом обработки. Для проекта это означает, что static metadata нельзя выдавать за всю runtime-историю.

Как это отражено в проекте: Проект отражает это разделением static и dynamic: static layer говорит о заявленной PE/unwind карте, dynamic layer наблюдает события процесса. Отчет показывает оба слоя отдельно, а итоговая оценка коррелирует их только при наличии общих оснований.

Что не надо завышать: Проект не сохраняет полный CPU CONTEXT каждого исключения и не строит полный crash-dump style route. Это нормально: он анализирует инфраструктуру unwind/handler, а не заменяет отладчик.

Глава 2: PE, секции, RVA/VA/file offset (стр. 17-25 и 108-165)

Смысл для анализатора: Теория требует сначала построить надежную карту файла. Нельзя читать .pdata, пока не проверены headers, section table, SizeOfImage, SizeOfHeaders, Data Directory и правила перевода RVA.

Как это отражено в проекте: src/pe.c реализует mandatory parse layer, structural checks, section mapping и headers mapping. src/util.c дает безопасные bounded reads, чтобы поврежденный файл не заставил parser читать за границами буфера.

Что не надо завышать: Проект не делает полный universal PE validator. Он проверяет те PE-инварианты, которые нужны для EH/unwind анализа: координаты, секции, Exception Directory и доступность bytes.

Глава 3: .pdata, RUNTIME_FUNCTION, .xdata (стр. 26-33 и 166-241)

Смысл для анализатора: Эта глава - ядро static части: RUNTIME_FUNCTION связывает диапазон кода с unwind metadata. Begin/End/UnwindInfoAddress являются RVA, а сама запись является контрактом, а не набором чисел.

Как это отражено в проекте: src/unwind.c читает массив RUNTIME_FUNCTION, src/heuristics.c проверяет диапазоны, сортировку, пересечения, duplicate begin и UnwindInfoAddress. Отчет показывает table entries и ошибки по каждой записи.

Что не надо завышать: Проект не пытается доказывать, что каждый BeginAddress совпадает с границей функции из дизассемблера. Теория сама предупреждает, что compiler/decompiler view может отличаться.

Глава 4: prolog, epilog, UNWIND_CODE и ABI (стр. 34-42 и 242-416)

Смысл для анализатора: UNWIND_INFO описывает эффект prolog: выделение стека, сохранение nonvolatile registers, frame pointer и специальные machine frames. Для анализатора важно не только распарсить bytes, но и понять declared effect.

Как это отражено в проекте: unwind_parse знает основные UWOP, проверяет длину operands, считает fixed_stack_bytes и masks saved registers. Это уже больше, чем синтаксический parser: он превращает bytes в модель заявленного поведения кадра.

Что не надо завышать: Автоматического сравнения с реальными инструкциями prolog нет. Это честно указано как граница: для observed model нужен дизассемблер и отдельные правила matching.

Глава 5: RtlLookupFunctionEntry и RtlVirtualUnwind (стр. 43-50 и 417-543)

Смысл для анализатора: Windows поднимается к caller через поиск FunctionEntry по ControlPc и применение UNWIND_INFO. Если таблица или metadata повреждены, stack walking и exception dispatch могут стать неверными.

Как это отражено в проекте: Проект проверяет те structures, от которых зависит RtlLookupFunctionEntry/RtlVirtualUnwind: function ranges, sort/overlap, UnwindInfoAddress, grammar UNWIND_INFO, handler flags and CHAININFO.

Что не надо завышать: Проект не вызывает RtlVirtualUnwind по живому стеку и не строит цепочку кадров. Он проверяет корректность карты, по которой такой unwind должен работать.

Глава 6: фазы exception handling и static/dynamic границы (стр. 51-59 и 544-721)

Смысл для анализатора: Search phase, unwind phase, VEH, SEH, UEF и C++ exceptions показывают, что часть информации существует только во время выполнения.

Как это отражено в проекте: Dynamic collector наблюдает first/second chance exceptions, handler hits, VEH/VCH/UEF registrations and hits. Static layer сохраняется отдельно, потому что он описывает файл, а не runtime route.

Что не надо завышать: Проект не интерпретирует всю C++ runtime semantics. Он фиксирует Windows-level events and handler metadata.

Глава 7: практическая ручная модель анализа (стр. 723-866)

Смысл для анализатора: Теория учит идти по слоям: machine point, PE image, function table, unwind info, handler layer, stack-walking result. Это напрямую влияет на структуру отчета.

Как это отражено в проекте: HTML report построен как evidence map: PE image, structural checks, sections, findings, dynamic validation, runtime function table. Finding объясняет rule, observed, expected and impact.

Что не надо завышать: Последний слой 'stack walking result' реализован не как полный автоматический stack walker, а как проверка предпосылок и runtime observations.

Глава 8: аномалии, обфускация, dynamic function tables (стр. 867-1035)

Смысл для анализатора: Теория описывает не только broken structures, но и suspicious patterns: reuse, dominance, unusual placement, dynamic tables, JIT и runtime-only metadata.

Как это отражено в проекте: src/heuristics.c реализует reuse/dominance/placement/combo findings. src/dynamic_win.c наблюдает RtlAddFunctionTable/RtlInstallFunctionTableCallback.

Что не надо завышать: Проект не реконструирует все JIT tables из памяти и не выполняет callback protocol. Он фиксирует регистрацию как runtime evidence.

Глава 9: архитектура анализатора и evidence map (стр. 1036-1151)

Смысл для анализатора: Эта глава прямо говорит, что настоящий analyzer не шумит warnings, а строит доказательную карту: откуда взялся факт, почему ему можно доверять, где заканчивается уверенность.

Как это отражено в проекте: Архитектура проекта соответствует этому: util/pe/unwind создают facts, heuristics превращает facts в findings, report объясняет findings, dynamic добавляет runtime observations.

Что не надо завышать: Observed model на уровне дизассемблированного prolog оставлен ручной проверке, поэтому документ не должен заявлять 'полная семантическая верификация кода'.

Глава 10: validation, corpus, testing (стр. 1152-1223)

Смысл для анализатора: Теория требует не только написать parser, но и проверять его на нормальных, поврежденных, статических, динамических и смешанных samples. Нужны negative cases, expected findings and regression discipline.

Как это отражено в проекте: В проекте есть tests, binaries for testing, reports, JSON/HTML outputs и режимы static/dynamic/mixed. Сами reports помогают сверять expected evidence.

Что не надо завышать: Текущий документ не утверждает, что corpus исчерпывает все возможные packers/JIT/runtime cases. Это учебно-исследовательский набор, который можно расширять.

Глава 11: архитектура EH/Unwind Analyzer (стр. 1224-1481)

Смысл для анализатора: Теория переходит от механики Windows к конкретному проекту: какие модули нужны, как они связаны, какие reports выдаются, как пользоваться режимами.

Как это отражено в проекте: Файлы проекта соответствуют этим блокам: util.c, pe.c, unwind.c, heuristics.c, dynamic_win.c, report.c, main.c. CLI flags позволяют включать нужные слои.

Что не надо завышать: Некоторые идеи главы 11 описаны как желаемая инженерная полнота, а не обязательный минимальный scope. Поэтому документ отделяет реализованные функции от future work.

Глава 12: каталог static findings (стр. 1482-1495)

Смысл для анализатора: Глава 12 задает логику 27 findings: evidence families, criteria, почему именно такой набор и как его защищать.

Как это отражено в проекте: 27 findings реализованы в heuristics.c, объясняются в report.c и перечислены ниже в отдельной матрице.

Что не надо завышать: Число 27 не закрывает все возможные будущие классы анализа. Оно закрывает текущую область PE32+ x64 EH/unwind integrity.

4. Каждое требование из матрицы подробно

Ниже матрица раскрыта словами. Смысл этого раздела - показать не только 'есть/нет', а как именно требование живет в программе: что оно защищает, какое действие выполняет код и какой результат получает пользователь.

T01. Не доверять входному PE как истине

Зачем это нужно: Файл считается источником заявлений, которые надо проверять по границам, размерам и связям. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: bounded reads, PE signature/header checks, fatal errors для неподдерживаемой основы. Кодовая привязка: src/util.c:41-68; src/pe.c:35-146. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T02. Разделять file, image и live module

Зачем это нужно: RVA, VA и file offset нельзя смешивать; static читает файл, dynamic работает с actual ImageBase процесса. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: static RVA->file offset; dynamic base_main + RVA. Кодовая привязка: src/pe.c:264-299; src/dynamic_win.c:789-803. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T03. Поддерживать именно Windows x64 PE32+

Зачем это нужно: Теория EH/unwind относится к x64 модели с .pdata/.xdata и 32-bit RVA внутри PE32+. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано как осознанный scope, PE32/x86 не смешивается с x64 unwind. Кодовая привязка: include/pe.h:18-19; src/pe.c:67-88. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T04. Сначала пройти mandatory PE layer

Зачем это нужно: DOS header, e_lfanew, NT signature, COFF, Optional Header, Magic, SizeOfImage, SizeOfHeaders. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано; при фатальных нарушениях дальнейший анализ не строится. Кодовая привязка: src/pe.c:35-146. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T05. Проверять Data Directory count

Зачем это нужно: Exception Directory нельзя читать, если NumberOfRvaAndSizes не покрывает нужный индекс. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: недостаточный directory count фиксируется как parse problem. Кодовая привязка: src/pe.c:121-134. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T06. Корректно понимать отсутствующий Exception Directory

Зачем это нужно: VA=0 и Size=0 - отсутствие таблицы; частично нулевая пара подозрительна. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: zero/zero не авария, partial zero - malformed. Кодовая привязка: src/pe.c:230-235; src/unwind.c:623-626. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T07. Сопоставлять Exception Directory с image и файлом

Зачем это нужно: Directory RVA/Size должны быть внутри SizeOfImage и мапиться в реальные bytes файла для static чтения. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: OOB/mapping/size checks до чтения entries. Кодовая привязка: src/pe.c:230-235; src/unwind.c:631-647. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T08. Проверять section table как основу координат

Зачем это нужно: Raw/virtual ranges, overlaps, alignment и SizeOfImage определяют доверие к дальнейшим RVA. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса

может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: первые 6 findings покрывают PE layout. Кодовая привязка: src/pe.c:174-228; src/heuristics.c:789-829. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T09. Учитывать headers и zero-fill

Зачем это нужно: RVA может попадать в headers; virtual tail может существовать в памяти, но не в файле. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: headers мапятся отдельно, virtual tail не читается как raw bytes. Кодовая привязка: src/pe.c:276-299. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T10. Читать RUNTIME_FUNCTION как три little-endian DWORD

Зачем это нужно: BeginAddress, EndAddress, UnwindInfoAddress - RVA, запись 12 байт. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: чтение DWORD bounded и little-endian через util. Кодовая привязка: src/util.c:53-68; src/unwind.c:657-659. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T11. Проверять размер Exception Directory

Зачем это нужно: Size должен давать целое число RUNTIME_FUNCTION entries; некратность 12 - тревожный сигнал. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: too small / not multiple становятся parse issue. Кодовая привязка: src/unwind.c:639-647. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T12. Проверять диапазоны функций

Зачем это нужно: BeginAddress < EndAddress, границы должны быть внутри image, таблица должна быть однозначной. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: invalid_ranges, rf_oob, rf_unsorted, rf_overlaps, duplicate_begin. Кодовая привязка: src/heuristics.c:612-779; src/heuristics.c:850-887. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T13. Не считать gaps автоматической ошибкой

Зачем это нужно: Дыры между RUNTIME_FUNCTION могут быть leaf code или padding. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано осторожно: gaps не являются отдельным finding. Кодовая привязка: src/heuristics.c:612-779. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T14. Проверять UnwindInfoAddress

Зачем это нужно: Это RVA на UNWIND_INFO; он должен быть внутри image и мапиться в файл при static чтении. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: unwind_rva_oob и parse failure. Кодовая привязка: src/unwind.c:55-68; src/heuristics.c:865-868. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T15. Разбирать UNWIND_INFO по grammar Windows x64

Зачем это нужно: Version, Flags, CountOfCodes, slots, operands, padding, tail должны читаться без усечения. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: parser всех основных UWOP и ошибок усечения. Кодовая привязка: src/unwind.c:164-604. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T16. Различать handler flags и CHAININFO

Зачем это нужно: EHANDLER/UHANDLER ведут к handler RVA; CHAININFO ведет к chained RUNTIME_FUNCTION. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: конфликт flags, chain tail, handler tail. Кодовая привязка: src/unwind.c:215-220; src/unwind.c:522-548. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T17. Строить declared model stack effects

Зачем это нужно: UNWIND_CODE должен давать смысл: stack allocation, saved nonvolatile registers, machine frame. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Частично реализовано: считает fixed_stack_bytes и masks; полного disasm сравнения нет. Кодовая привязка: src/unwind.c:274-520. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T18. Сравнивать metadata placement с ролью секций

Зачем это нужно: Unwind metadata обычно read-only/non-executable, handler target должен быть executable. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано static по section characteristics, не runtime page protections. Кодовая привязка: src/heuristics.c:117-153; src/heuristics.c:966-997. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T19. Искать reuse/dominance patterns

Зачем это нужно: Массовое повторение UNWIND_INFO/handler targets может быть обфускационным или packer-like сигналом. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано через ratios, thresholds, dominant RVAs. Кодовая привязка: src/heuristics.c:681-787; src/heuristics.c:909-955. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и

не делать уверенный вывод там, где основа сломана.

T20. Использовать correlation, а не одиночный шум

Зачем это нужно: Сильнее не отдельный слабый сигнал, а связка table/unwind/placement/reuse/dynamic. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: combo findings и static/dynamic correlation. Кодовая привязка: src/heuristics.c:1004-1023; src/report.c:507-518. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T21. Объяснять finding как evidence

Зачем это нужно: Finding должен говорить где найдено, что нарушено, что наблюдалось, как должно быть и почему важно. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: Rule/Observed/Expected/Impact в HTML/JSON. Кодовая привязка: src/report.c:65-195; src/report.c:1347-1365. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T22. Не делать malware verdict

Зачем это нужно: Теория говорит о suspicious evidence, а не о доказательстве вредоносности. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: suspicion score/confidence, не антивирусный verdict. Кодовая привязка: src/heuristics.c:503-587; docs/evidence_matrix.md. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T23. Наблюдать runtime handler route

Зачем это нужно: Dynamic должен увидеть регистрации, handler hits, exceptions и function table registrations. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано через Windows Debug API и breakpoints. Кодовая привязка: include/dynamic.h:3-13; src/dynamic_win.c:656-1195. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T24. Учитывать dynamic function tables/JIT

Зачем это нужно: Static не увидит таблицы, созданные в памяти; dynamic должен хотя бы фиксировать регистрацию. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Частично реализовано: фиксируются API calls, но не реконструируются все remote entries. Кодовая привязка: src/dynamic_win.c:569-573; src/dynamic_win.c:886-1029. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T25. Показывать runtime evidence отдельно от static score

Зачем это нужно: Динамика не тот же score 0..100; это observed runtime signals и runtime evidence level. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: dynamic_signals, runtime_evidence_level, final assessment. Кодовая привязка: src/report.c:486-604; src/report.c:1825-1885. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T26. Поддерживать static-only, dynamic-only, mixed запуск

Зачем это нужно: Разные режимы нужны для файла на диске, runtime проверки и объединенного evidence. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Реализовано: --heuristics, --dynamic и совместное использование. Кодовая привязка: src/main.c:191-242; src/main.c:389-408. Это означает, что требование не осталось текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

T27. Оставлять ручную проверку там, где нужна семантика кода

Зачем это нужно: Prolog disassembly, C++ scope tables и полный stack walk шире текущей реализации. Если эту дисциплину убрать, analyzer начинает строить вывод на непроверенных заявлениях файла или на перепутанных координатах. В теме EH/unwind это особенно опасно, потому что одна ошибка на уровне адреса может увести parser к чужим bytes и породить ложные findings.

Как это делает проект: Честно оформлено: report дает manual validation steps, а не делает ложный all-seeing вывод. Кодовая привязка: src/report.c:624-677; src/report.c:1807-1822. Это означает, что требование не осталось

текстовой идеей из теории, а имеет конкретное место в реализации.

Какой вывод/действие следует: если это mandatory layer, программа прекращает анализ при фатальной ошибке. Если это diagnostic layer, проект сохраняет счетчик, finding, evidence или mitigation и продолжает, чтобы собрать полную картину. Такое разделение защищает от двух крайностей: не падать на каждом подозрительном поле и не делать уверенный вывод там, где основа сломана.

5. Полная пошаговая работа проекта от запуска до отчета

Этот раздел отвечает на вопрос 'как проект проверяет бинарник полностью'. Не просто какие признаки есть, а в каком порядке программа идет по файлу, какие шаги обязательны, какие включаются флагами, какое решение принимается и почему.

#	Шаг	Когда выполняется	Код	Что происходит и зачем
1	CLI parse	Всегда	main.c:191-242	Читает input path и flags. Решает, будет ли heuristics, dynamic, JSON, HTML, dump, ops, timeout.
2	Open/read file	Всегда	util.c:12-35	Файл загружается в Buf. Если файл нельзя прочитать, анализ невозможен.
3	Bounded read discipline	Всегда	util.c:41-68	Все дальнейшие чтения идут через функции, которые проверяют off+size <= buffer size.
4	DOS header/e_lfanew	Всегда	pe.c:10-18	Проверяется MZ и смещение до NT headers.
5	PE signature	Всегда	pe.c:51-54	Проверяется PE\0\0; без этого нельзя считать файл PE.
6	COFF header	Всегда	pe.c:60-76	Читаются Machine, NumberOfSections, SizeOfOptionalHeader; AMD64 обязателен.
7	Optional Header Magic	Всегда	pe.c:83-88	0x20b подтверждает PE32+. PE32/x86 не входит в scope.
8	Image fields	Всегда	pe.c:100-115	Читаются ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders.
9	DataDirectory count	Всегда	pe.c:121-134	Exception Directory читается только если NumberOfRvaAndSizes позволяет.
10	Section headers	Всегда	pe.c:151-172	Читаются name, VirtualSize, VirtualAddress, raw size/ptr, characteristics.
11	Section structural validation	Всегда	pe.c:174-228	Считаются raw/virtual OOB, overlaps, misalignment.
12	Exception directory OOB	Всегда	pe.c:230-235	Проверяется, не выходит ли directory range за SizeOfImage, нет ли partial zero pair.
13	Read runtime function table	Всегда после PE parse	unwind.c:610-671	Zero/zero means absent; otherwise RVA maps to file offset and Size must be valid.
14	Exception RVA -> file offset	Если directory есть	pe.c:276-299; unwind.c:631-647	Нельзя читать RVA как offset; parser ищет section and raw bytes.
15	Read each RUNTIME_FUNCTION	Если table valid	unwind.c:657-659	Три little-endian DWORD: BeginAddress, EndAddress, UnwindInfoAddress.
16	Prepare UnwindOptions	Всегда при RF entries	main.c:344-352	Опции strict_abi, follow_chain, max_chain_depth влияют на parse strictness.
17	Parse each UNWIND_INFO	Для каждой RF	main.c:353-356; unwind.c:164-604	Разбирается header, flags, unwind codes, operands, chain/handler tail.
18	Record parse success/error	Для каждой RF	main.c:355-376	us_ok фиксирует, можно ли доверять decoded summary.
19	Optional dump/ops	Только --dump/--ops	main.c:357-376	Печатает entries и decoded operations для ручной проверки.
20	Run heuristics	Только --heuristics	main.c:389-397; heuristics.c:589-1038	Считает 27 static findings, score, verdict, confidence, mitigations.
21	PE findings	Только --heuristics	heuristics.c:789-829	Первые findings возникают из section/Exception Directory structural counters.
22	RF findings	Только --heuristics	heuristics.c:850-887	Invalid ranges, OOB, unsorted, overlaps, duplicates.
23	Unwind findings	Только --heuristics	heuristics.c:894-904	Parse failures and anomalies from UNWIND_INFO.
24	Reuse/dominance findings	Только --heuristics	heuristics.c:909-955	Dominant UnwindInfoAddress/handler patterns.
25	Placement findings	Только --heuristics	heuristics.c:966-997	Unwind metadata and handler targets section permissions/roles.
26	Combo findings	Только --heuristics	heuristics.c:1004-1023	Correlation inside static layer increases strength.
27	Run dynamic collector	Только --dynamic	main.c:403-408; dynamic_win.c:656-1195	Запускает target as debuggee and collects runtime evidence.
28	CreateProcess debug	Только --dynamic	dynamic_win.c:699-711	Target запускается под Debug API with timeout.
29	Get actual ImageBase	Только --dynamic	dynamic_win.c:789-803	CREATE_PROCESS_DEBUG_EVENT дает base_main; handler breakpoints use base_main + RVA.
30	Set API breakpoints	Только --dynamic	dynamic_win.c:560-573	VEH/VCH/UEF and dynamic function table APIs become observation points.
31	Set handler breakpoints	Только --dynamic + parsed handlers	dynamic_win.c:727-803	Handlers from UNWIND_INFO become runtime breakpoints.
32	Process debug events	Только --dynamic	dynamic_win.c:748-852	Handles load/unload/exit/exception/breakpoint/single-step.

#	Шаг	Когда выполняется	Код	Что происходит и зачем
33	Record registrations/hits	Только --dynamic	dynamic_win.c:886-1029	Records VEH/VCH/UEF, RtlAddFunctionTable, RtlInstallFunctionTableCallback.
34	Build assessment summary	Если report has hr/dr	report.c:486-604	Counts dynamic signals, runtime evidence level, static/dynamic correlation, final confidence.
35	Write JSON	Только --json	main.c:488-490; report.c:781-1133	Machine-readable report for tests and reproducibility.
36	Write HTML	Только --html	main.c:502-504; report.c:1542-1951	Human-readable evidence dashboard.
37	Show findings evidence	Если --heuristics	report.c:1347-1365; report.c:1740-1822	Each finding gets rule/observed/expected/impact and analysis details.
38	Show dynamic evidence	Если --dynamic	report.c:1825-1885	Cards and event table show runtime observations and log.
39	Show RUNTIME_FUNCTION table	Всегда в HTML	report.c:1888-1923	Every entry has Begin/End/Unwind RVA/section/status/flags/stack/handler/errors.
40	Exit code/result	Всегда	main.c:428-509	Failures to write outputs affect exit_code; parse failures stop earlier.

Обязательные и опциональные слои

Всегда выполняется базовый parse route: чтение файла, PE headers, section table, Exception Directory, RUNTIME_FUNCTION и попытка разобрать UNWIND_INFO. Это нужно даже для dynamic-only запуска, потому что dynamic collector использует результаты static pre-pass: handler RVA, rf_count, us/us_ok и PE map. Без этого он не смог бы поставить breakpoints на handlers из UNWIND_INFO.

Опционально выполняется heuristics layer. Он включается флагом --heuristics и именно он считает 27 findings, score, static verdict и confidence. Без --heuristics проект может распарсить structures и сформировать report, но не будет полноценной статической оценки подозрительности.

Опционально выполняется dynamic layer. Он включается флагом --dynamic и требует Windows Debug API. Этот слой запускает target и наблюдает события выполнения. Он не обязан находить статические findings: иногда файл статически обычный, но во время выполнения регистрирует dynamic function table или VEH. И наоборот, файл может быть статически подозрительным, но конкретный запуск не пройдет по handler route.

Смешанный режим получается, когда пользователь включает и --heuristics, и --dynamic. Тогда отчет содержит две группы доказательств и final assessment. В этом режиме особенно важна correlation: если static finding и runtime event относятся к одному смысловому слою, confidence повышается. Если они независимы, report не притворяется, что динамика подтвердила статику.

6. Подробный разбор блоков работы проекта

Дальше идут не тезисы, а разбор по слоям. Каждый слой отвечает на пять вопросов: что говорит теория, что обязан делать анализатор, как проект делает это в коде, какой результат/решение получается и что проект честно не обещает.

1. Runtime-сцена исключения и границы проекта

Что говорит теория: Главы 1 и 6 объясняют, что исключение - это не просто 'ошибка', а событие в конкретном потоке с RIP, RSP, CONTEXT, ExceptionRecord, search phase и unwind phase. Из этого следует требование к анализатору: он не должен делать вид, что одна таблица в PE полностью описывает весь runtime маршрут. Static слой видит metadata. Dynamic слой может увидеть часть реального маршрута. Полный смысл появляется только при разведении этих уровней.

Что делает проект: В проекте это разведено архитектурно. Static path читает файл на диске и строит PE/unwind model. Dynamic path запускает target через Windows Debug API и собирает DynamicReport. В main.c сначала выполняется общий static pre-pass, а затем, если включен --dynamic, вызывается dynamic_collect_windows. Это видно в src/main.c:294-408. Dynamic module прямо описан как 'Windows-only dynamic evidence collector' и ограничивает задачу наблюдением handler hits, registrations, dynamic function tables и first/second chance, не взломом защит.

Как это работает технически: Когда пользователь запускает только static, проект не знает реальный RIP при исключении. Он может сказать: 'если RIP попадет в этот диапазон, такая-то RUNTIME_FUNCTION должна вести к такой-то UNWIND_INFO'. Когда включена динамика, проект получает actual base module из CREATE_PROCESS_DEBUG_EVENT, ставит breakpoints на handler RVAs и API регистрации, затем записывает события. Поэтому динамический слой не заменяет static: он отвечает на другой вопрос - что реально произошло при запуске.

Где в коде: include/dynamic.h:3-13; src/main.c:294-408; src/dynamic_win.c:656-1195; src/report.c:486-604

Граница реализации: Проект не является полноценным debugger/crash dump analyzer. Он не восстанавливает полный call stack через RtlVirtualUnwind для каждого исключения и не анализирует все registers. Это не ошибка, а граница ВКР: цель - integrity/suspicion analysis EH/unwind metadata плюс runtime evidence, а не замена WinDbg.

2. PE как карта: headers, sections, Data Directory

Что говорит теория: Главы 2, 9 и 11 требуют начинать не с .pdata, а с доверенной минимальной PE-карты. Нужно проверить DOS header, e_lfanew, PE signature, COFF header, Machine, Optional Header Magic, SizeOfImage, SizeOfHeaders, NumberOfRvaAndSizes, section table. Идея теории проста: PE-файл не является истиной, он только заявляет о себе набор фактов, которые parser обязан проверять перед тем, как на них опираться.

Что делает проект: Проект делает это в src/pe.c. pe_parse читает MZ/e_lfanew, PE signature, File Header, Optional Header, проверяет AMD64 и PE32+ magic. Затем читает ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders, NumberOfRvaAndSizes и Exception Directory. Структура PeImage сохраняет не только поля, но и counters structural anomalies, чтобы позже heuristics превратил их в findings.

Как это работает технически: Если ломается mandatory layer - например файл не PE, architecture не AMD64, Optional Header слишком мал - анализ прекращается, потому что дальше нельзя безопасно интерпретировать байты. Если же файл формально читается, но section table странная, проект не останавливается сразу. Он фиксирует raw_oob, virtual_oob, overlaps, misalignment, exception_directory_oob и продолжает настолько, насколько возможно. Это важное инженерное различие: fatal parse error и suspicious structure - разные результаты.

Где в коде: src/pe.c:35-146; src/pe.c:174-255; include/pe.h:18-19; src/heuristics.c:605-609; src/heuristics.c:789-829

Граница реализации: Проект специально не поддерживает PE32/x86. Для комиссии это нужно объяснять так: ВКР исследует Windows x64 EH/unwind, где основная модель строится через PE32+ .pdata/.xdata и RUNTIME_FUNCTION. Добавление x86 SEH было бы другой моделью, а не 'маленьким расширением'.

3. RVA, VA, file offset, headers и zero-fill

Что говорит теория: Большая часть теории постоянно возвращается к трем координатным системам. RVA - смещение внутри image, VA - runtime address после прибавления ImageBase, file offset - позиция байта в файле на диске. Static parser не может читать по VA и не должен считать RVA file offset. Для чтения bytes он обязан перевести RVA через section table и проверить, что bytes реально существуют в raw data.

Что делает проект: pe_find_section_for_rva и pe_rva_to_fileoff реализуют эту дисциплину. Они ищут section по virtual span, учитывают raw_size и отдельно разрешают RVA внутри SizeOfHeaders. Если RVA попадает в virtual область, но delta >= raw_size, чтение из файла запрещается: loader в памяти может дать zero-fill, но в файле этих bytes нет. Это прямо закрывает те тонкости, о которых пользователь спрашивал: VirtualSize нужен для virtual layout, SizeOfRawData - для чтения с диска.

Как это работает технически: Например, UnwindInfoAddress=0x6200 не читается как offset 0x6200. Проект сначала ищет section, где лежит RVA 0x6200, затем считает PointerToRawData + (RVA - VirtualAddress). Если section говорит, что эта часть существует только виртуально, parser не подставляет нули и не придумывает UNWIND_INFO. Он возвращает mapping failure, который дальше становится parse failure или finding. В динамике другая формула: actual base_main + RVA, потому что процесс уже загрузил image.

Где в коде: src/pe.c:264-299; src/unwind.c:55-68; src/unwind.c:631-647; src/dynamic_win.c:789-803

Граница реализации: Static модель не читает память процесса; dynamic модель не заменяет static file parser. Поэтому в отчете важно смотреть, из какого мира взято поле: file offset - для анализа файла, VA/address - для runtime event, RVA - общий язык между ними.

4. Exception Directory и чтение RUNTIME_FUNCTION

Что говорит теория: Теория говорит: Exception Directory в Optional Header дает RVA и Size. Для x64 эта область обычно является массивом RUNTIME_FUNCTION. Каждая запись занимает 12 bytes: BeginAddress, EndAddress, UnwindInfoAddress, все три - DWORD/RVA. Size должен быть кратен 12, иначе неясно, сколько целых записей содержится в каталоге.

Что делает проект: unwind_read_runtime_functions_ex обрабатывает все базовые случаи. Если RVA=0 и Size=0, таблица отсутствует и это не crash. Если одно поле нулевое, а другое нет - malformed. Если directory RVA не мапится в файл, если размер выходит за файл, меньше 12 или не кратен 12, чтение не выполняется как нормальное. Если все проверки прошли, проект читает каждую запись через buf_read_u32.

Как это работает технически: Это не 'просто три числа'. После чтения entries heuristics проверяет их как карту функций: BeginAddress < EndAddress, диапазоны внутри SizeOfImage, порядок по BeginAddress, отсутствие пересечений, отсутствие duplicate_begin. Эти проверки нужны потому, что RtlLookupFunctionEntry ищет запись по ControlPc; если карта неоднозначна, весь unwind route может быть неверным.

Где в коде: src/unwind.c:610-671; src/util.c:53-68; src/heuristics.c:612-779; src/heuristics.c:850-887

Граница реализации: Проект не пытается 'починить' таблицу. Он фиксирует evidence. Это правильнее для analyzer: ремонт без понимания кода может сделать таблицу еще менее правдивой.

5. UNWIND_INFO, UWOP и declared stack model

Что говорит теория: Главы 4, 5, 7, 8 и 9 объясняют, что UNWIND_INFO - это не исполняемый код, а рецепт, по которому Windows может отменить эффект prolog и получить caller context. Нужно читать Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset, массив UNWIND_CODE, выравнивание и tail. Tail меняет смысл в зависимости от flags: handler RVA, language-specific data или CHAININFO.

Что делает проект: unwind_parse читает header, проверяет version/flags, проходит по CountOfCodes, знает slots для основных UWOP, читает operands, считает fixed_stack_bytes и masks saved registers, проверяет truncated operands, конфликт CHAININFO с EHANDLER/UHANDLER, читает chained RUNTIME_FUNCTION или handler_rva. Ошибки сохраняются в UnwindSummary и затем участвуют в findings unwind_parse_fail и unwind_anomalies.

Как это работает технически: Технически parser не просто двигает offset. Каждый opcode имеет свою длину в slots. UWOP_ALLOC_LARGE, SAVE_NONVOL, SAVE_XMM и FAR variants требуют дополнительных slots. Если CountOfCodes говорит, что slots есть, а bytes в файле закончились, проект не продолжает чтение наугад. Если handler flag установлен, parser после aligned unwind code array читает handler RVA. Если CHAININFO установлен, tail читается как еще одна RUNTIME_FUNCTION.

Где в коде: src/unwind.c:23-52; src/unwind.c:164-604; src/report.c:1888-1923

Граница реализации: Главное частичное место: проект строит declared model из metadata, но не строит полноценный observed model через дизассемблирование prolog и сравнение push/sub/mov с UWOP. В отчете для этого предусмотрены manual validation steps. На защите это нужно называть future work, а не скрытой возможностью.

6. Handler layer и language-specific tail

Что говорит теория: Теория отделяет handler layer от обычной function table. EHANDLER/UHANDLER говорят, что за UNWIND_CODE после padding лежит handler RVA и далее language-specific data. Handler target должен быть адресом исполняемого обработчика, а не произвольной data area. Но полный смысл C++/SEH scope tables зависит от compiler/runtime ABI.

Что делает проект: Проект читает handler_rva, проверяет, что он не выходит за SizeOfImage, определяет section handler target и права section. Findings handler_nonexec_section и handler_writable_section появляются, если target не попадает в executable область или находится в writable section. В dynamic режиме проект ставит breakpoints на handler_rva, найденные из UNWIND_INFO, и может увидеть unwind_handler_hit.

Как это работает технически: Static вывод: 'metadata заявляет handler RVA'. Dynamic вывод: 'этот handler реально получил управление'. Это разные уровни. Если handler RVA в .rdata, это подозрительно по static placement. Если при запуске breakpoint на handler сработал, это уже runtime confirmation, что route не только заявлен, но и использован.

Где в коде: src/unwind.c:540-548; src/heuristics.c:715-724; src/heuristics.c:989-997; src/dynamic_win.c:727-803; src/report.c:1400-1415

Граница реализации: Проект не декодирует полностью C++ EH tables, scope table, type descriptors и compiler-specific exception data. Это отдельная крупная тема. ВКР ограничивается Windows x64 unwind/handler placement и runtime observation.

7. Placement, permissions и роль секций

Что говорит теория: Теория объясняет, что права read/write/execute важны для интерпретации. .text обычно executable, .rdata read-only, .data writable, .pdata/.xdata - metadata, которая должна быть доступна для чтения, но обычно не должна исполняться. Это не абсолютный закон по имени секции, но сильная инженерная норма.

Что делает проект: Проект не доверяет только имени секции. Он смотрит section characteristics: IMAGE_SCN_MEM_EXECUTE и IMAGE_SCN_MEM_WRITE. Для UnwindInfoAddress он считает unwind_in_exec_section, unwind_unusual_section, unwind_writable_section. Для handler target - handler_nonexec_section и handler_writable_section. Затем combo findings усиливают вывод, если странное размещение metadata совпадает со странным handler placement.

Как это работает технически: Если UNWIND_INFO лежит в executable section, это не автоматически 'вирус', но это нарушает ожидаемую роль metadata. Если handler target лежит в non-executable/unknown section, это сильнее: handler должен быть кодом, которому Windows может передать управление. Если оба факта встречаются вместе, это уже корреляция, а не одиночная странность.

Где в коде: src/heuristics.c:117-153; src/heuristics.c:966-1023; src/report.c:52-64; src/report.c:1696-1730

Граница реализации: Проверка статическая: она смотрит PE section characteristics, а не реальные page protections после loader, packer unpacking или VirtualProtect. Runtime memory protection audit в проект не входит.

8. Reuse, dominance, thresholds и score

Что говорит теория: Главы 8, 9 и 12 подводят к мысли, что подозрительность часто рождается не из одного сломанного поля, а из паттерна: один UnwindInfoAddress используется слишком многими функциями, один handler доминирует, placement и integrity findings усиливают друг друга. Теория не задает чисел +25 или threshold 0.30; она задает критерии, какие отношения считать важными.

Что делает проект: heuristics.c задает reproducible engineering scale: score constants, thresholds, confidence rules. RUNTIME table/integrity errors получают больший вес, weak placement - меньший, correlation - дополнительный вес. Verdict thresholds: >=70 high, >=40 moderate, >=15 low. Confidence растет от числа families и strong/medium

evidence, но снижается при single signal, small sample, placement-only и single family.

Как это работает технически: Почему не +47? Потому что веса не являются физической величиной. Их назначает разработчик как прозрачную экспертную шкалу. Важно, что порядок силы обоснован: OOB и parse failures опаснее, чем необычное имя секции; combo stronger than isolated weak signal; small sample снижает уверенность. Проект оставляет числа в коде как проверяемую и изменяемую модель, а не как магию.

Где в коде: src/heuristics.c:31-84; src/heuristics.c:503-587; src/heuristics.c:909-1023; src/report.c:565-604

Граница реализации: Score не является статистически откалиброванной вероятностью вредоносности. Это эвристическая шкала suspicious evidence. На комиссии нужно говорить именно так.

9. Dynamic function tables, JIT и runtime evidence

Что говорит теория: Разделы 3.2.29 и глава 8 объясняют, что RUNTIME_FUNCTION существует не только в .pdata. JIT/runtime может создать dynamic function table в памяти и зарегистрировать ее через RtlAddFunctionTable или RtlInstallFunctionTableCallback. Static parser файла этого не увидит.

Что делает проект: Dynamic collector ставит breakpoints на RtlAddFunctionTable и RtlInstallFunctionTableCallback, а также на AddVectoredExceptionHandler, AddVectoredContinueHandler и SetUnhandledExceptionFilter. Когда target вызывает эти API, проект записывает событие с module/thread/address/arguments. Это закрывает требование 'не ограничиваться только .pdata', но на уровне наблюдения регистрации.

Как это работает технически: Если binary создает таблицу во время выполнения, static report может быть чистым, потому что на диске этой таблицы нет. Dynamic report покажет rtl_add_function_table или rtl_install_function_table_callback. Это не значит, что проект полностью прочитал remote table entries и проверил каждую запись; он фиксирует факт runtime registration и аргументы, достаточные как evidence для аналитика.

Где в коде: include/dynamic.h:7-13; src/dynamic_win.c:569-573; src/dynamic_win.c:886-1029; src/report.c:1383-1415

Граница реализации: Полная реконструкция JIT dynamic tables в удаленном процессе, чтение callback-generated entries и проверка их UNWIND_INFO - будущая работа. Проект работает и с non-JIT бинарниками, и с бинарниками, которые регистрируют dynamic tables, но не является JIT reverse engineering framework.

10. Report как доказательная карта, а не декоративный HTML

Что говорит теория: Глава 9 формулирует сильное требование: finding должен быть маленьким расследованием. Он должен сохранять связь между наблюдением, правилом, ожидаемым состоянием, влиянием на unwind/analysis и ограничением уверенности. Иначе отчет превращается в набор warning без доказательств.

Что делает проект: report.c формирует JSON и HTML. Для findings есть rule, observed, expected, impact; для location есть representative_rva, file_offset, section, characteristics, related_function. Для dynamic events есть тип, module, thread, exception, chance, address, args и смысл. Для итогов есть static score, dynamic signals, final confidence, static/dynamic correlation.

Как это работает технически: Отчет нужно читать сверху вниз: верхние карточки дают обзор, PE image и sections дают coordinate system, structural check показывает базовые нарушения, findings объясняют статические доказательства, dynamic section показывает runtime observations, RUNTIME_FUNCTION table дает низкоуровневую проверку каждой записи. Верхний блок частично дублирует summary не случайно: он дает быстрый dashboard, а ниже идут доказательства.

Где в коде: src/report.c:65-195; src/report.c:781-1133; src/report.c:1347-1536; src/report.c:1542-1951

Граница реализации: HTML не должен восприниматься как 'автоматический суд'. Он является интерфейсом evidence. Если finding спорный, аналитик должен открыть указанные RVA/file offset и проверить вручную.

7. Особо важный пример 3.2.45: медленный проход одной записи

Пункт 3.2.45 важен именно потому, что показывает правильный стиль мышления. Нельзя сказать: 'я прочитал три DWORD и закончил'. Нужно понять, что BeginAddress, EndAddress и UnwindInfoAddress являются RVA, проверить их роль, связать с секциями, открыть metadata, посмотреть declared prolog effect и только потом говорить, насколько запись согласована.

Шаг из теории	Как должен работать анализатор	Что делает проект	Честная оценка
1. Прочитать Begin/End/UnwindInfoAddress как RVA	Не путать с VA и file offset; учитывать little-endian DWORD.	buf_read_u32 + RuntimeFunction в src/unwind.c:657-659.	Реализовано.
2. Для static перевести UnwindInfoAddress в file offset	Найти section, проверить raw bytes, не читать zero-fill.	pe_rva_to_fileoff в src/pe.c:276-299; вызов в src/unwind.c:55-68.	Реализовано.
3. Проверить Begin/End как code range	Begin < End, range внутри image, таблица сортирована и не пересекается.	heuristics.c:612-779 и findings invalid_ranges/rf_oob/rf_unsorted/rf_overlaps.	Реализовано структурно.
4. Проверить UnwindInfoAddress как metadata	UNWIND_INFO должен быть доступен, структурно разбираем и лежать в разумной metadata области.	unwind_parse + placement findings в heuristics.c:966-981.	Реализовано для metadata structure/placement.
5. Сравнить unwind codes с реальным prolog	Дизассемблировать BeginAddress и сравнить push/sub/mov с UWOP.	Проект выводит данные и manual validation step, но не disassembler matching.	Частично / future work.
6. Связать handler route	Если есть EHANDLER/UHANDLER, проверить handler RVA и route.	Static handler placement + dynamic breakpoint on handler RVA.	Реализовано частично: route наблюдается только при runtime срабатывании.

На комиссии это нужно формулировать так: проект реализует автоматическую structural/evidence часть примера 3.2.45 - чтение, координаты, границы, placement, UNWIND_INFO parse, handler observation. Полная семантическая проверка 'metadata совпадает с реальными инструкциями prolog' требует встроенного дизассемблера и отдельного набора правил ABI; она осознанно оставлена как развитие, чтобы ВКР не заявляла больше, чем реально проверяет.

8. 27 статических findings: почему они покрывают сердце проекта

Число 27 не взято из стандарта Microsoft и не является магическим. Оно получилось из coverage заявленной области: PE layout, Exception Directory, function table integrity, UNWIND_INFO integrity, metadata/handler placement, reuse/dominance и correlation rules. Если убрать любую группу, анализатор теряет отдельный слой доказательств. Если добавить новые группы, это уже расширение scope - например disassembly/prolog matching или полный decoder C++ EH data.

#	Finding	Семейство	Как проверяется	Почему важно
1	pe_section_raw_oob	PE layout	PointerToRawData + SizeOfRawData > file_size	Такой raw range нельзя безопасно читать; все RVA->file offset выводы становятся сомнительными.
2	pe_section_virtual_oob	PE layout	VirtualAddress + max(VirtualSize, SizeOfRawData) > SizeOfImage	Секция заявляет область вне загруженного image; это ломает карту памяти PE.
3	pe_section_raw_overlap	PE layout	интервалы [raw_ptr, raw_ptr+raw_size) двух секций пересеклись	Один участок файла имеет двух владельцев; parser может прочитать данные не той секции.
4	pe_section_virtual_overlap	PE layout	интервалы RVA двух секций пересеклись	Один RVA становится неоднозначным; нельзя надежно понять роль области.
5	pe_section_misaligned	PE layout	FileAlignment, SectionAlignment, SizeOfHeaders, SizeOfImage, ImageBase или порядок секций нарушают инварианты	Loader contract выглядит поврежденным или нестандартным.
6	pe_exception_directory_oob	Exception Directory	RVA/Size частично нулевые, диапазон выходит за SizeOfImage, не мапится в file offset или выходит за file size	.pdata нельзя корректно прочитать.
7	no_runtime_functions	function table	Exception Directory отсутствует или дает 0 записей при наличии ожидаемого кода	Может быть нормой для leaf-only/resource-only image, но для нетривиального x64 кода требует проверки.
8	invalid_ranges	function table	положительный диапазон функции не выполняется	Запись не описывает функцию, lookup/unwind становится некорректным.
9	rf_oob	function table	begin/end RVA вне image	Function table ссылается на код, которого нет в загруженном образе.
10	unwind_rva_oob	function table	unwind_rva == 0, >= SizeOfImage или pe_rva_to_fileoff false	Windows/анализатор не сможет прочитать metadata функции.
11	rf_unsorted	function table	BeginAddress идет не по возрастанию	Бинарный поиск по ControlPc может работать неоднозначно или неверно.
12	rf_overlaps	function table	после сортировки previous.End > next.Begin	Один адрес кода принадлежит нескольким функциям.
13	duplicate_begin	function table	две записи имеют одинаковый BeginAddress	Нельзя однозначно выбрать unwind metadata для начала функции.
14	unwind_parse_fail	UNWIND_INFO	не читается header, codes array, operands, chain tail или handler tail	Virtual unwind может оборваться или восстановить неверный context.
15	unwind_anomalies	UNWIND_INFO	version/flags/opcode/operand/order/ABI/chain anomalies	Metadata похожа на поврежденную, нестандартную или специально сформированную.
16	unwind_reuse_strong	reuse	reuse ratio выше high threshold	Может быть dispatcher/protector-like map, особенно если подтверждается другими слоями.
17	handler_dominance	handler	много RUNTIME_FUNCTION ссылаются на один handler target	Похоже на централизованную обработку исключений, VM/protector/runtime dispatcher.
18	unwind_reuse_moderate	reuse	reuse ниже strong, но совпадает с другими статическими signals	Сам по себе слабее, но с соседними аномалиями становится evidence.
19	unwind_in_exec_section	placement	section characteristics содержит EXECUTE	Metadata обычно не должна исполняться как код.
20	unwind_unusual_section	placement	section не похожа на .xdata/.rdata/.pdata и роль неочевидна	Нужна ручная проверка linker/packer/runtime причины.
21	unwind_writable_section	placement	section characteristics содержит WRITE	Unwind map обычно стабильная read-only metadata; writable повышает риск подмены.
22	handler_nonexec_section	handler placement	handler_rva не попадает в executable section	Handler должен быть кодом; non-exec target ломает dispatch model.

#	Finding	Семейство	Как проверяется	Почему важно
23	handler_writable_section	handler placement	handler section имеет WRITE	Исполняемый handler в writable области нетипичен и требует объяснения.
24	placement_exec_nonexec_handler_combo	correlation	unwind_in_exec_section и handler_nonexec_section встречаются вместе	Два placement-сигнала усиливают друг друга.
25	placement_integrity_combo	correlation	metadata placement совпадает с parse/OOB/table integrity	Разные слои указывают на одну проблемную область.
26	reuse_dominance_combo	correlation	unwind reuse совпадает с handler dominance	Похоже на centralized dispatcher pattern.
27	writable_metadata_handler_combo	correlation	writable UNWIND_INFO и writable handler target встречаются вместе	Вместе это сильнее, чем два отдельных слабых наблюдения.

Как finding превращается в решение программы

Проект не останавливается на большинстве findings. Он собирает evidence, добавляет score, определяет severity/family, сохраняет representative RVA и потом продолжает анализ, чтобы не потерять другие признаки. Исключение - fatal parse layer: если файл невозможно прочитать как поддерживаемый PE32+ AMD64, дальнейшие unwind выводы были бы ложными, поэтому main.c завершает анализ с parse error. Это разделение критично: analyzer должен быть устойчивым к поврежденным PE, но не должен строить фантазии на невалидной основе.

Каждый finding в отчете должен отвечать на вопросы: где найдено, какое правило нарушено, что наблюдалось, как должно быть и почему это влияет на unwind/analysis. В проекте это сделано в report.c: finding_rule_text, finding_observed_text, finding_expected_text и finding_impact_text, а HTML выводит evidence block по каждому finding.

9. Динамические сигналы: что проект видит при запуске

Динамические сигналы нельзя оценивать как те же 27 static findings. Static finding говорит о свойствах файла и metadata. Dynamic signal говорит о факте, который произошел в живом процессе: исключение, регистрация handler-а, попадание в handler, регистрация dynamic function table. Поэтому динамика имеет signal count и runtime evidence level, а не отдельный score 0..100.

Сигнал	Что найдено	Точка наблюдения	Как интерпретировать
exception	Debug API увидел исключение target	EXCEPTION_DEBUG_EVENT, не breakpoint/single-step	Показывает, что runtime exception flow действительно возник; важны first/second chance.
unwind_handler_hit	исполнение дошло до handler из UNWIND_INFO	breakpoint на base_main + handler_rva	Сильнее регистрации: handler реально получил управление.
veh_registration	зарегистрирован VEH	breakpoint на AddVectoredExceptionHandler, RCX/RDX args	Показывает runtime слой исключений вне статической .pdata.
veh_hit / veh_handler_hit	VEH реально получил управление	breakpoint на зарегистрированный handler address	Подтверждает фактический маршрут обработки.
vch_registration	зарегистрирован VCH	breakpoint на AddVectoredContinueHandler	Показывает continue-handler layer.
vch_hit / vch_handler_hit	VCH реально получил управление	breakpoint на зарегистрированный VCH	Подтверждает runtime route после exception processing.
uef_registration	установлен UnhandledExceptionFilter	breakpoint на SetUnhandledExceptionFilter	Может быть обычной runtime-логикой или перехватом top-level unhandled exceptions.
uef_hit / uef_handler_hit	UEF реально получил управление	breakpoint на установленный filter	Сильный runtime signal для unhandled exception path.
rtl_add_function_table	runtime добавил function table	breakpoint на RtlAddFunctionTable, args RCX/RDX/R8	Статическая .pdata уже не полная карта unwind.
rtl_install_function_table_callback	runtime установил callback function table	breakpoint на RtlInstallFunctionTableCallback, args RCX/RDX/R8/R9	Возможен JIT/runtime/protector scenario.
process_exit	target завершился	EXIT_PROCESS_DEBUG_EVENT	Нужен для понимания полноты dynamic run и exit code.
timeout	сессия достигла лимита	timeout_ms в debug loop	Найденные события полезны, но отсутствие других событий не доказывает их отсутствия.

Почему dynamic может быть слабым, средним или сильным

report.c считает runtime_evidence_level так: handler hits или hits VEH/VCH/UEF дают strong component; RtlAddFunctionTable/RtlInstallFunctionTableCallback и second chance дают medium component; registrations и first chance дают weak component. Если dynamic session неполная, уровень снижается до weak, потому что часть событий могла быть не увидена. Это честная модель: наблюдение runtime важно, но неполная instrumentation не должна выглядеть как стопроцентное доказательство.

Смешанный режим сильнее, когда static и dynamic говорят об одном слое. Например, static нашел handler placement anomaly, а dynamic реально увидел handler hit. Или static нашел strange metadata placement, а dynamic увидел dynamic function table registration. Тогда report.c выставляет static_dynamic_correlation и повышает final confidence.

10. Режимы запуска и как читать отчет

Проект можно запускать в трех практических режимах. Во всех режимах есть базовый static pre-pass, потому что даже динамике нужны PE map, RUNTIME_FUNCTION и handler RVA. Разница в том, включается ли heuristics engine, dynamic collector и как report.c собирает итог.

```
# Только статическая оценка
.\artifacts\windows-x64\eh_unwind_analyzer.exe
  .\path\sample.exe
  --heuristics
  --json .\reports\static.json
  --html .\reports\static.html

# Только динамическое runtime-наблюдение без полноценного static score
.\artifacts\windows-x64\eh_unwind_analyzer.exe
  .\path\sample.exe
  --dynamic
  --timeout-ms 5000
  --json .\reports\dynamic.json
  --html .\reports\dynamic.html

# Смешанный режим: static findings + dynamic signals + correlation
.\artifacts\windows-x64\eh_unwind_analyzer.exe
  .\path\sample.exe
  --heuristics
  --dynamic
  --timeout-ms 5000
  --json .\reports\mixed.json
  --html .\reports\mixed.html
```

Элемент отчета	Откуда берется	Как правильно понимать
RUNTIME_FUNCTION records	rf_count из Exception Directory	Сколько 12-byte entries удалось прочитать из .pdata/Exception Directory.
Ошибки разбора	us_ok / parse_error	Сколько UNWIND_INFO не удалось разобрать как валидную x64 unwind structure.
Аномальные записи	UnwindSummary anomalies	Сколько entries имеют подозрительные flags/opcodes/chain/operands.
Эвристические признаки	hr.finding_count	Сколько из 27 static findings сработало.
Статическая оценка	hr.score	Оценка только статического слоя; не является итогом всей динамики.
Итоговый вывод	assessment_summary	Объединяет static score, runtime evidence level, correlation и confidence.
Статика / динамика	hr.evidence_level + runtime_evidence_level	Сравнение силы доказательств двух независимых слоев.
Динамические сигналы	dynamic_signal_count	Сколько классов runtime events реально наблюдалось.
Итоговая уверенность	final_confidence	Confidence after static/dynamic/correlation, не вероятность malware.
PE image	PeImage	ImageBase, SizeOfImage, SizeOfHeaders, Exception Directory и alignments.
Structural check	PeImage counters	Первые проверки доверия к карте секций и Exception Directory.
Sections	Section Table	VirtualAddress, VirtualSize, raw offset/size, rights; база для placement выводов.
Findings table	HeuristicFinding[]	Главная доказательная часть: rule, observed, expected, impact.
Dynamic validation	DynamicReport	Runtime events: exceptions, handler hits, registrations, dynamic tables.
Journal	dr.details	Сырая timeline отладочного collector-а, полезная для ручной проверки.
RUNTIME_FUNCTION table	rf + us + us_ok	Низкоуровневый список функций, unwind RVA, flags, stack, handler, errors.

Если вверху 'Динамическая проверка: завершена', это значит, что collector запустился и дошел до завершения. Это не означает, что dynamic дала высокий подозрительный verdict. Для динамики нужно смотреть карточку dynamic signals, runtime evidence level, таблицу dynamic validation и журнал. Если runtime events есть, но static score нулевой, итог может быть runtime_signals_only. Если static и dynamic коррелируют, итог становится сильнее.

11. Что из теории не реализовано полностью и как это защищать

Область теории	Почему это шире текущей ВКР	Как честно сказать комиссии
Полный RtlVirtualUnwind stack walking	Нужно читать live CONTEXT, память стека, module tables и повторять unwind по кадрам.	Проект проверяет metadata, от которой зависит stack walking, но не заменяет WinDbg/DbgHelp stack walker.
Prolog disassembly matching	Нужен дизассемблер x64 и правила сравнения observed instructions с declared UWOP.	Проект декодирует declared model и дает адреса для ручной проверки; automated prolog matcher - развитие.
Полный C++ EH decoder	C++ scope tables/type descriptors compiler-specific и выходят за общий Windows unwind слой.	ВКР анализирует Windows x64 EH/unwind metadata и handler placement, а не весь MSVC C++ ABI.
Runtime page protection audit	Нужно читать VirtualQueryEx/память процесса и учитывать изменения после loader/packer.	Static смотрит section characteristics; dynamic собирает events, но не делает полный memory permission scanner.
Полная реконструкция JIT tables	Callback table может генерироваться по запросу; нужно читать remote memory/callback protocol.	Проект фиксирует регистрацию RtlAddFunctionTable/RtlInstallFunctionTableCallback как runtime evidence.
PE32/x86 SEH	Другая exception model, другие структуры, нет той же x64 .pdata/.xdata логики.	Ограничение PE32+ AMD64 сделано сознательно, чтобы выводы были корректны.

Ключевая защитная формулировка

Проект не претендует на роль универсального антивируса, отладчика или полного reverse engineering suite. Его вклад - специализированный analyzer для Windows x64 EH/unwind metadata, который безопасно читает PE32+ файл, проверяет карту секций и Exception Directory, читает RUNTIME_FUNCTION, разбирает UNWIND_INFO, оценивает 27 static findings, наблюдает runtime exception/unwind evidence через Debug API и формирует объяснимый HTML/JSON отчет. Теория шире проекта, но центральная цепочка, нужная для заявленной ВКР, реализована и документирована.

12. Итоговое заключение

После повторного чтения всей теории вывод такой: проект покрывает центральную реализуемую часть теоретической модели - от недоверенного PE-файла до evidence-based отчета по Windows x64 EH/unwind metadata. Он учитывает не только главу про RUNTIME_FUNCTION, но и PE parsing, section validation, адресные пространства, UNWIND_INFO grammar, handler layer, static/dynamic границы, dynamic function table registrations, scoring/confidence, report evidence и ограничения. Там, где теория требует более широкой системы - полный stack walker, disassembler matcher, C++ EH decoder, JIT table reconstruction - это прямо обозначено как граница, а не замаскировано под реализацию.

Такой ответ защищает проект честно: он сложен не количеством if/else, а тем, что связывает несколько несовместимых наивно миров - файл на диске, виртуальный PE image, runtime module, таблицы функций, unwind metadata, handler route, static heuristics и dynamic events - и сохраняет доказательную цепочку для пользователя.